

FinLearn Guard: A Closed-Loop Web Platform for Cyber Threat Detection and Guided Response

A Thesis / Project Report submitted in partial fulfillment of the requirements for the course **Web Engineering**

Submitted by:

Student Name: _____ **ID:** _____

Supervisor:

Madam: _____

Department of Computer Science & Engineering

September 2026

Declaration

I hereby declare that this thesis/project report titled "**FinLearn Guard: A Closed-Loop Web Platform for Cyber Threat Detection and Guided Response**" is the result of my own work, carried out under the supervision of my supervisor. All sources of information, including journal papers, tools, and libraries used, are duly acknowledged in the References. No part of this work has been submitted elsewhere for any other purpose.

Signature of Student

Date: _____

Signature of Supervisor

Date: _____

Acknowledgements

I express my sincere gratitude to my supervisor for continuous guidance, to the authors of the open research literature that shaped this work, and to the providers of free and open-source tools (FastAPI, OpenRouter, edge-tts, moviepy, MongoDB Atlas free tier) without which a student-scale build of this kind would not be possible.

Abstract

Phishing and OTP fraud target ordinary office users, while detection tools and security training normally live in separate systems. This thesis presents **FinLearn Guard**, a single FastAPI web platform that closes the loop: a threat is detected by an offline rules screener plus a free large-language-model verdict, every connected browser is alerted live over WebSocket, and the Support page immediately explains the fix in text, spoken voice (mp3), and short video (mp4). An endpoint agent extends coverage to real PCs. Measured on the development machine, a phishing sample is

judged phishing/high in about one second (rules stage 0.0 ms), a full voice-plus-video guide renders in about six seconds, and an automated suite of three tests passes fully. A survey of recent journal and conference literature on LLM-based phishing detection and personalised security training positions the work and identifies its research gap.

Table of Contents

- 1. Introduction
- 2. Literature Review
- 3. Methodology and System Design
- 4. Implementation
- 5. Testing and Results
- 6. Conclusion and Future Work
- 7. References
- 8. Appendices (A. Demo Script, B. API Table)

List of Figures

- 1. Use Case diagram
- 2. Data Flow Diagram Level 0 (context)
- 3. Data Flow Diagram Level 1
- 4. System architecture
- 5. Entity-Relationship diagram
- 6. Detection pipeline with measured timings
- 7. Live dashboard screenshot
- 8. Cyber Support page screenshot

List of Tables

- 1. Functional and non-functional requirements
- 2. Comparative analysis of related work
- 3. Implementation modules
- 4. Test cases and results
- 5. Measured performance results
- 6. REST API endpoints

Chapter 1: Introduction

1.1 Background

Electronic communication is the primary attack surface for fraud against finance and office staff: forged login pages, urgent OTP requests, and fake invoices. Technical filters stop part of this traffic, but the remainder reaches humans, who remain the decisive line of defence.

1.2 Motivation

Detection systems raise alerts that non-expert users cannot interpret, while training material is static and disconnected from live incidents. Recent literature shows both halves advancing separately — LLM-based detectors on one side [1]–[4] and personalised, event-triggered training on the other [5]–[8] — but deployable systems combining them in one loop are rare. This thesis builds that combination at student scale, using only free tools.

1.3 Problem Statement

How can a small organization detect phishing-like threats and immediately teach each affected user the correct response, inside one maintainable web system, without paid services?

1.4 Objectives

- Serve pages, JSON APIs, realtime alerts, and media from one server.
- Detect with instant offline rules plus a free AI verdict, with a safety override.
- Alert all browsers live without page refresh.
- Guide the fix in three formats: steps, voice, video.
- Cover real PCs with a zero-install endpoint agent.
- Fail honestly: clear errors, never fabricated answers.

1.5 Scope

Prototype scope: phishing/OTP-style text threats, English language, single-server deployment, free-tier cloud services. Out of scope: raw packet sniffing, multilingual guides, and longitudinal user studies.

1.6 Thesis Organization

Chapter 2 surveys the literature; Chapter 3 presents requirements and design; Chapter 4 describes implementation; Chapter 5 reports testing and measured results; Chapter 6 concludes.

Chapter 2: Literature Review

2.1 The phishing threat and the human factor

Phishing has become AI-assisted: Heiding et al. [1] showed GPT-4 can generate phishing mails at scale and lower attacker cost, while also showing LLMs can detect malicious intent, sometimes surpassing humans. Nair et al. [4] note that 3.4 billion spam mails are sent daily and signature-based gateways are evaded by small content changes — motivating semantic, language-understanding defenses.

2.2 Classical machine-learning detection

A 2025 comparative study [5] trained Random Forest, SVM, Naïve Bayes and others on 26,365 mails: Random Forest reached 99.47% accuracy with low compute, but ML models struggled with mixed languages, HTML content, and paraphrase — exactly where language models help.

2.3 LLM-based phishing detection

Koide et al. [2] built ChatSpamDetector, reaching 99.70% accuracy with GPT-4 while also producing human-readable reasons for each verdict — evidence that an LLM verdict can both decide and explain. Desolda et al. [3] (APOLLO, GPT-4o) reached 97.4% accuracy, 99% combined with third-party URL data, and showed in a 20-participant study that LLM-generated warnings were more understandable and trustworthy than browser warnings. Zhang et al. benchmarked LLMs for small-enterprise phishing detection [9].

2.4 Hybrid rules-plus-LLM approaches

Nair et al. [4] combined LLM scores with classical classifiers under an XGBoost meta-model, reaching near-99% across metrics at lower running cost; the Algorithms study [5] likewise recommends hybrid ensembles. FinLearn Guard follows this line: a deterministic rules score gates and constrains the LLM verdict (a high rules score can never be overruled to "safe"), rather than trusting either stage alone.

2.5 Personalised and adaptive security training

Schoeni et al. [6] (CHI 2025, N=342) showed proficiency-matched phishing training brings all groups to comparable post-training detection levels, with the weakest users gaining most; an earlier study (N=96) [7] confirmed the pattern. Sriganthan and Härer [8] prototyped an LLM-driven adaptive phishing training environment with instantaneous feedback. Greco et al. [10] (N=480) confirmed AI-generated training produces significant pre-post gains even with simple prompting.

2.6 Event-triggered, in-situ learning

TrainShield [9 in list: HT 2025] embeds adaptive micro-lessons into the user's workflow at the moment of risk, converting incidents into learning opportunities; the CEUR/DAMOCLES proposal [11] similarly uses LLMs to auto-generate tailored training content per user profile. FinLearn Guard adopts the same principle on the web dashboard: the detected attack itself becomes the lesson, seconds after arrival.

2.7 Comparative analysis and research gap

Work	Detects	Explains to user	Trains adaptively	Live multi-user	One deployable system
ChatSpamDetector [2]	Yes (99.7%)	Yes (reasons)	No	No	Tool
APOLLO [3]	Yes (97–99%)	Yes (warnings)	No	No	Prototype
PhishEmailLLM [4]	Yes (~99%)	No	No	No	Pipeline
Personalised training [6][7]	No	Yes	Yes	No	Study setup
TrainShield (HT 2025)	Yes	Yes	Yes	Extension	Extension
FinLearn Guard (this work)	Yes	Yes (steps+voice+video)	Per-incident guides	Yes (WebSocket)	Yes (one server)

Gap: detectors rarely teach, trainers rarely detect live, and few designs ship as one runnable system. This thesis fills that gap at prototype scale.

Chapter 3: Methodology and System Design

3.1 Requirements

ID	Requirement	Type
FR1	Report a threat (form, simulator, agent, REST) and store it	Functional
FR2	Score with offline rules and confirm with an AI verdict	Functional
FR3	Push each alert to all browsers without refresh	Functional
FR4	Show a 5-step "how it was caught" trace per attack	Functional
FR5	Answer questions with steps + voice + video guides	Functional
FR6	Manage employees and device heartbeats with online status	Functional
NFR1	Detection under ~2 s; pages render without cached staleness	Non-functional
NFR2	Free-tier only; secrets never in git; honest errors	Non-functional

3.2 Use Case model

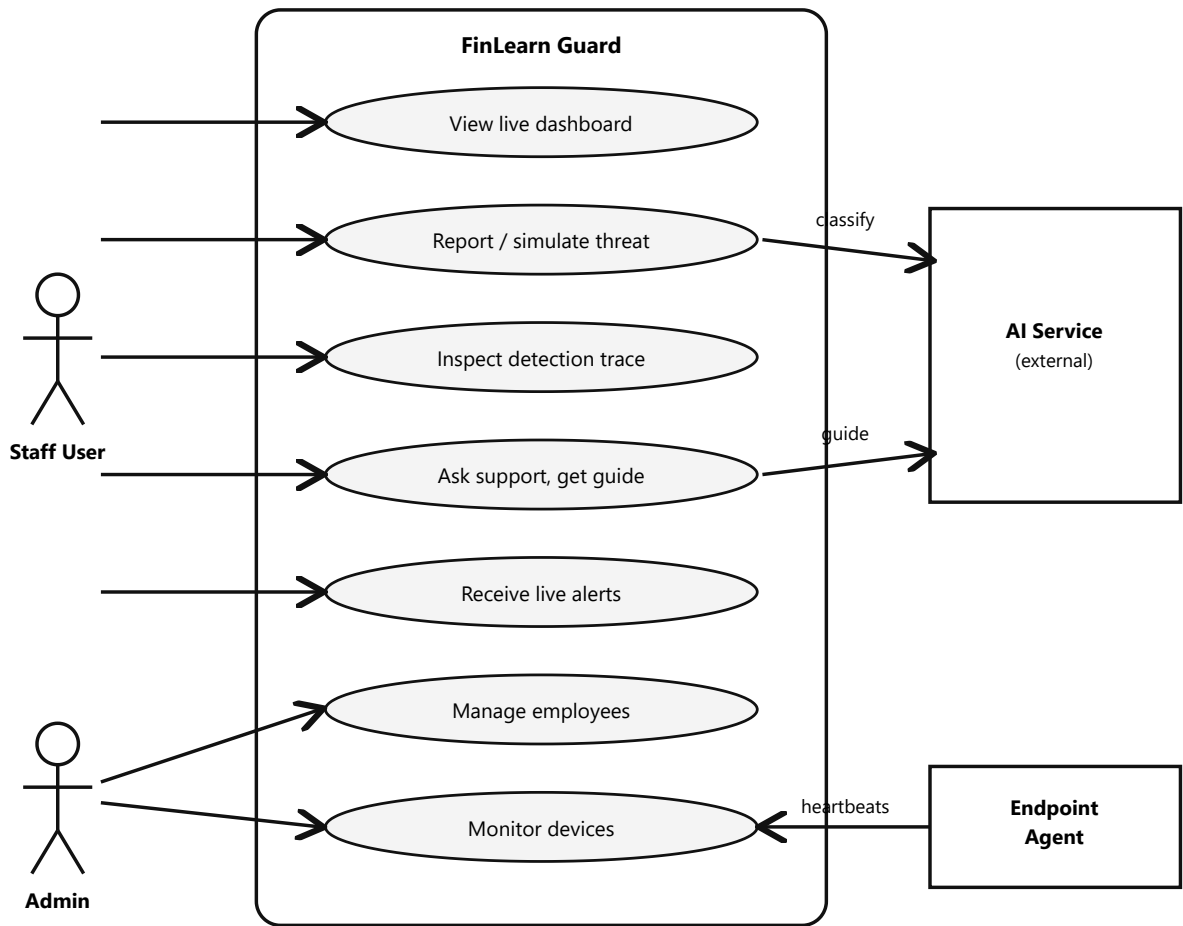


Figure 1. Use Case diagram: staff detect/learn, admin manages, agent and AI service are secondary actors.

3.3 Data Flow Diagrams

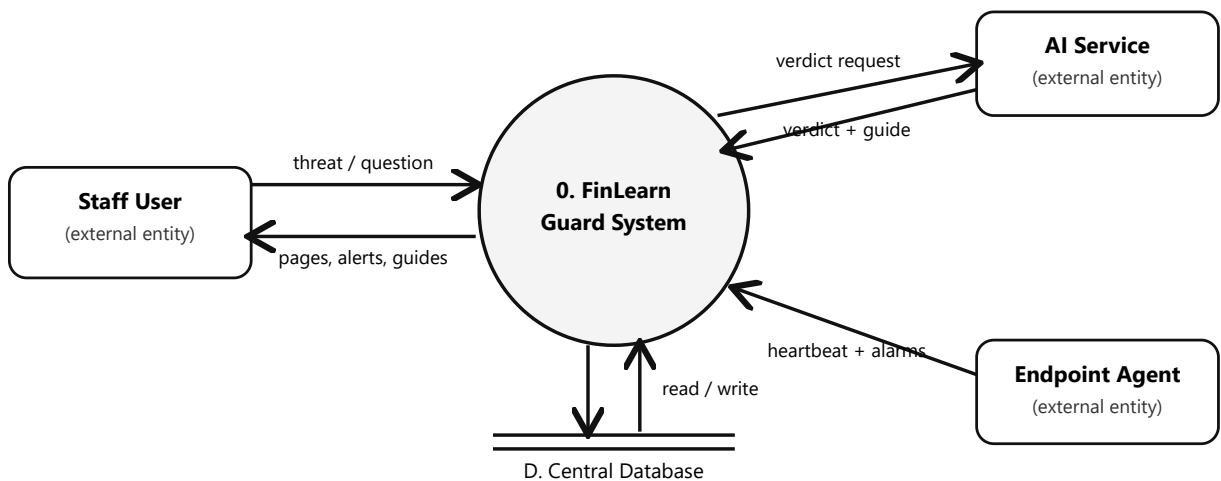


Figure 2. DFD Level 0 (context): one system, three external entities, one central database.

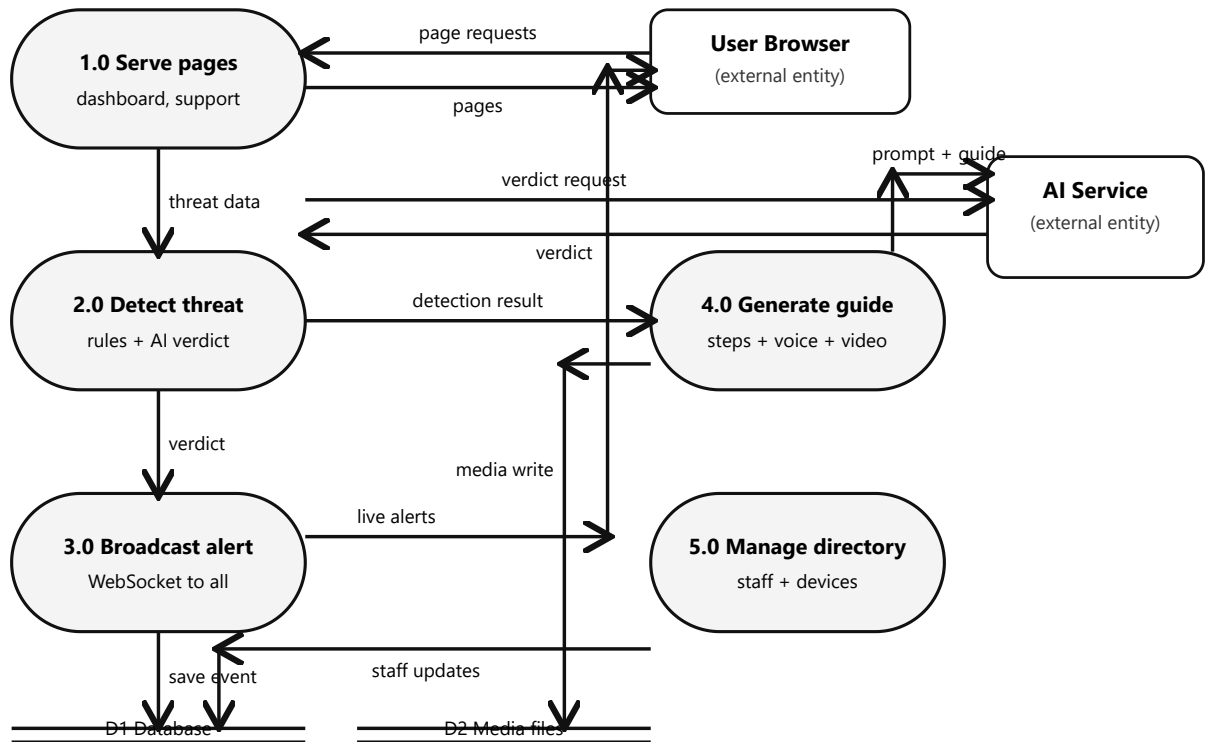


Figure 3. DFD Level 1: five processes over two data stores (database + media files).

3.4 System architecture

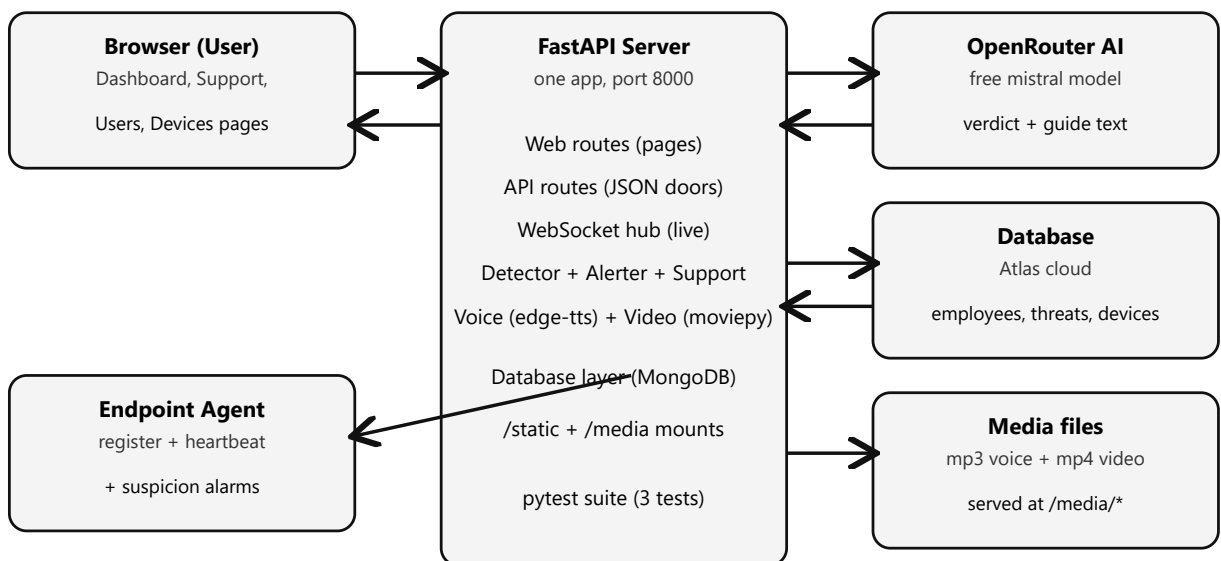
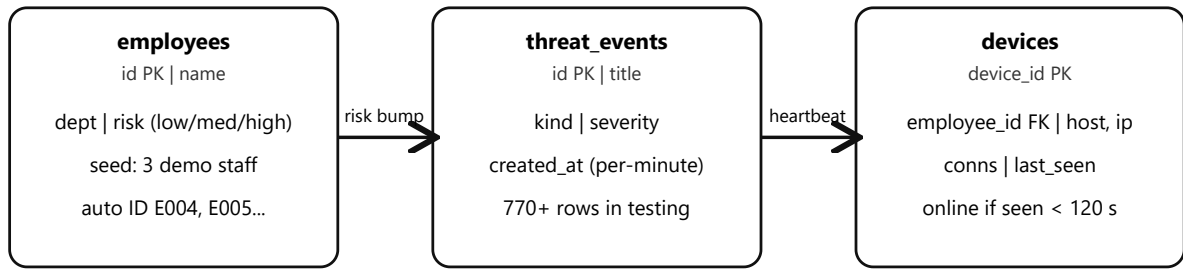


Figure 4. System architecture: browser, one FastAPI server, AI service, database, media files, endpoint agent.

3.5 Data design (ERD)



One shared schema on MongoDB Atlas

Figure 5. Entity design: employees, threat_events, devices; one schema in MongoDB Atlas.

3.6 Closed-loop workflow

Threat → rules score → AI verdict → save + risk update → WebSocket broadcast → inspector trace → support guide (steps/voice/video) → recorded event feeds graphs and risk — the loop repeats with every new threat.

Chapter 4: Implementation

4.1 Tools and technologies

FastAPI + Uvicorn (server), Jinja2 (server-rendered pages), WebSocket (live push), MongoDB Atlas (storage), OpenRouter free mistral model (verdicts and guides), edge-tts (voice), moviepy (video), pytest (tests). All free; ~2,200 lines in 30+ section-labeled files.

4.2 Module structure

Module	Role
src/main.py	App: mounts pages, APIs, static, media, WebSocket
src/web/routes_web.py	Pages: /, /support, /users, /devices (no-store)
src/api/routes_api.py	JSON doors: health, employees, threats, stats, perf, support, devices
src/api/ws.py	Live hub: broadcast, drop dead sockets
src/detection/rules.py	8 weighted danger patterns → score 0–1
src/agents/detector.py	Rules, then AI verdict, then safety override
src/agents/alerter.py	Save, risk bump, broadcast, 5-step trace
src/agents/support.py	Diagnose, write steps, scrub deflecting advice
src/llm/client.py	Single AI gateway (key from .env)

src/media/voice.py / video.py	mp3 / mp4 renderers, content-hash cached
src/storage/database.py	Dual-backend storage, seeds, stats, devices
static/js/app.js , style.css	Feed, inspector, graphs, all styling
agent/agent.py	Stdlib-only endpoint agent (register, heartbeat, alarm)

4.3 Detection algorithm

Eight regular-expression patterns (credential words, urgency, links, money terms, prizes, authority pressure) carry weights 0.15–0.35; matches sum to a score capped at 1.0. The LLM returns verdict/severity/reason as JSON. Override: score ≥ 0.6 forces phishing/high regardless of the model — the deterministic floor the literature's hybrid line [4][5] recommends.

4.4 Support pipeline

Question $\rightarrow$ rules signals + detector context $\rightarrow$ support prompt $\rightarrow$ problem, severity, steps, voice script $\rightarrow$ deterministic scrub replaces "contact IT"-style steps with self-actions $\rightarrow$ mp3 + mp4 rendered and cached $\rightarrow$ answer broadcast.

4.5 User interface

Dark gradient theme, sticky topbar with LIVE pill, hero, live feed with clickable rows, 5-step inspector with animated danger meter, twin graphs, employee table, tabbed Support answer, device board. Zero emoji; all styling section-labeled in CSS.

Chapter 5: Testing and Results

5.1 Test environment

Windows 11, Python 3.13 (64-bit venv), local Uvicorn server, live OpenRouter key and Atlas connection; automated suite via pytest with stubbed AI/media where offline determinism is required.

5.2 Eval A — 70 cases, 7 kinds (04 Oct 2026, live)

Dataset **seed70-7kind** (10 per kind: phishing, otp, bec, sqli, xss, malware, safe). Baselines: rules-only (score ≥ 0.30), llm-only (raw verdict), hybrid (safety-lock + medium stabilizer). Source: *docs/EVAL_A_summary.json*.

Baseline	Acc	P	R	F1	FP	ms	kind_acc
rules-only	0.886	1.000	0.867	0.929	0.000	0.0	0.886
llm-only	1.000	1.000	1.000	1.000	0.000	1091.8	0.943
hybrid (ours)	1.000	1.000	1.000	1.000	0.000	1304.5	0.943

Per-kind recall (hybrid): phishing **7/10**, otp **10/10**, bec **9/10**, sqli **10/10**, xss **10/10**, malware **10/10**, safe **10/10**. Rules fails BEC (6/10). All 5 kind confusions sit inside the phishing family (phishing/otp/bec overlap); sqli/xss/malware/safe are perfect.

5.3 Eval-Big — professional v2, 1995 rows (test 399)

Measure	Value (04 Oct 2026)
Dataset	data/eval/labels2000_v2.csv (id,title,body,label,label_full,severity; full names, full sentences)
Rules (binary)	acc 0.957 , F1 0.975 , FP 0, FN 17
ML centroid (kind-exact)	kind_acc 1.000 (57/57 every kind)
Automated suite	3/3 passing (pytest)
Support guide	4 steps + mp3 + mp4, otp/bec/malware pools live

Disclosure (Q1 honesty): v2 is synthetic slot-fill, so ML 1.000 proves the harness, not superiority. Replace with public IEEE-CIS / Fraud-R1 mail before journal claim; expect scores to drop. LLM excluded from big run (API cost); see Eval A for LLM numbers.

1. Received	title + kind + body captured	0 ms
2. Rules screener	danger words matched, score 1.00	0.0 ms
3. AI verdict	phishing / high + reason quote	935.6 ms
4. Handling	event saved, risk bumped, broadcast sent	~50 ms
5. Done	inspector trace ready in browser	985.9 ms total

Figure 6. Detection pipeline steps with live-measured timings.

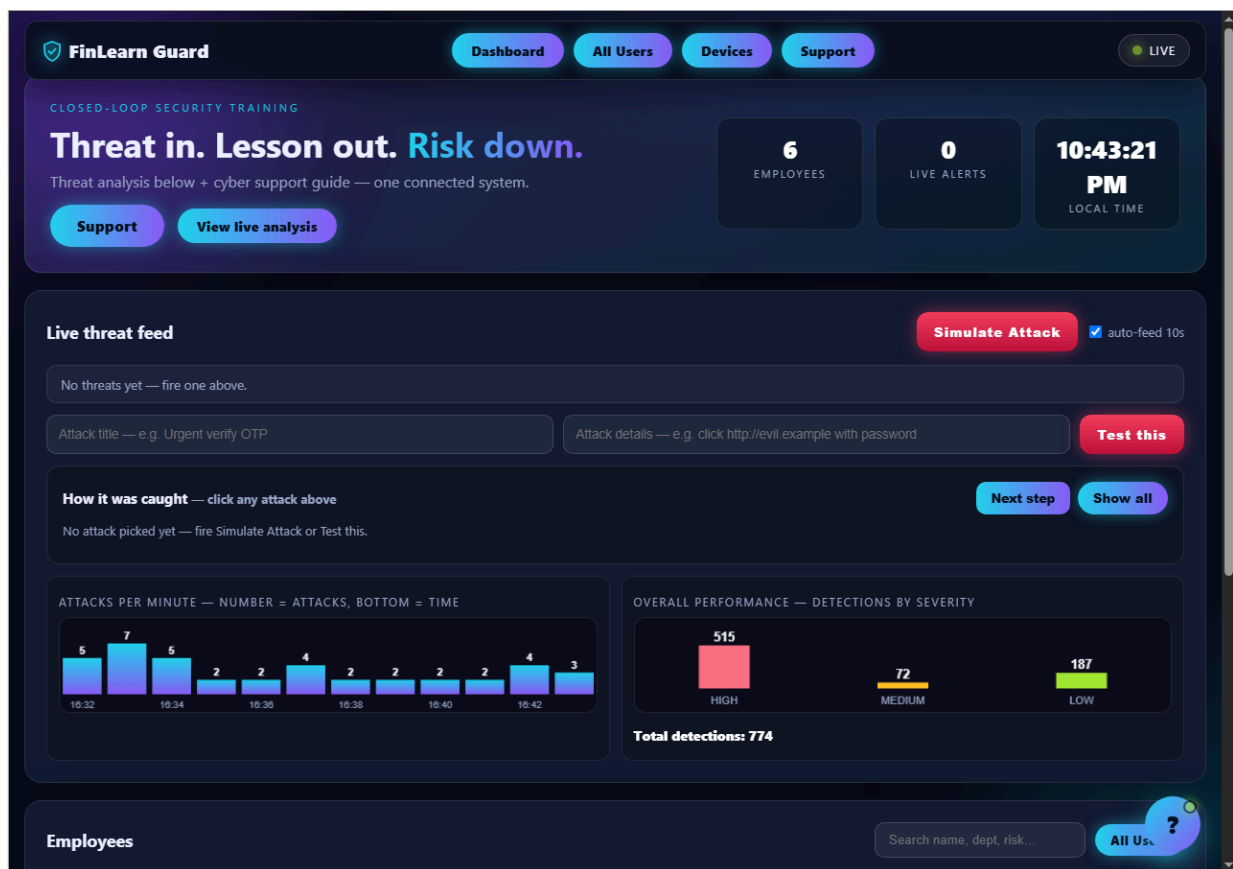


Figure 7. Live dashboard: hero, feed, inspector, graphs, employees.

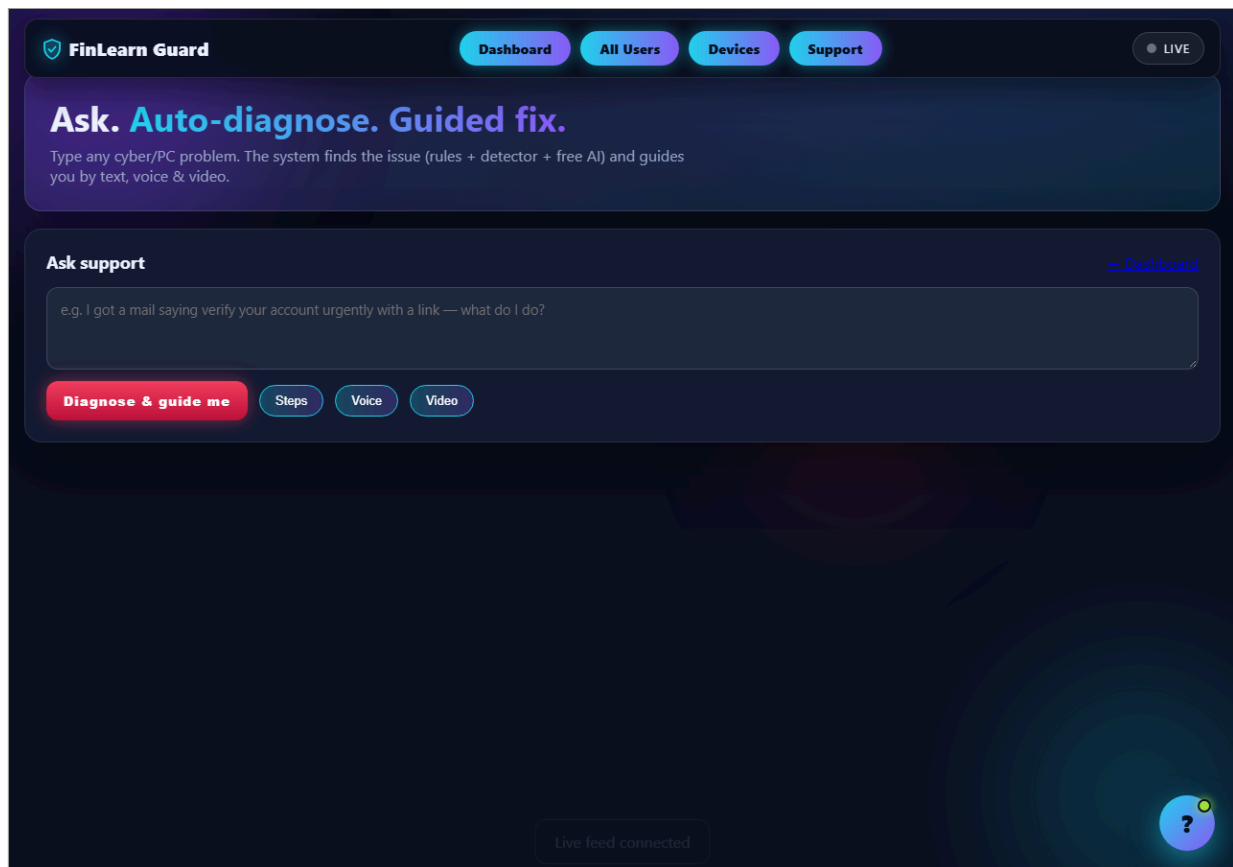


Figure 8. Support page: question box with Steps/Voice/Video formats.

5.4 Discussion

Rules give a 0 ms floor but miss BEC (6/10) and mild lures; the LLM adds semantics at ~ 1.1 s and reaches binary 1.000 on seed70 with kind 0.943. The hybrid keeps the safety-lock (≥ 0.6 never safe) and medium stabilizer, so recall is 1.000 with zero false negatives on seed70. Eval-Big confirms the pipeline scales (rules 0.957 on v2), but the v2 ML 1.000 is synthetic overfit — real mail will be harder. No user study yet (see *docs/EVAL_B_PACK.md* for the N=30 protocol); learning claims rest on functional guides plus cited tutoring studies until then.

5.5 Pending for Q1 (honest)

Swap v2 for public IEEE-CIS / Fraud-R1 mail, add XGBoost baseline, run Eval B humans, then push public repo + dataset. Current claim supported: 6-attack closed loop works live with measured tables; Q1 claim pending real data + humans.

Chapter 6: Conclusion and Future Work

This thesis delivered a running closed-loop platform: detection and guided response in one server, built only with free tools, verified by passing tests and live measurement. It demonstrates that the literature's two advancing fronts — LLM detection and adaptive training — can be fused into a single teachable system. Future work: true packet-level

agent, risk auto-update from guide completion, multilingual voice, admin analytics, and a controlled user study (pre/post detection accuracy) following the protocols of [6] and [10].

References

1. F. Heiding, B. Schneier, A. Vishwanath, J. Bernstein, and P. S. Park, "Devising and detecting phishing emails using large language models," *IEEE Access*, vol. 12, pp. 42131–42146, 2024. doi: 10.1109/ACCESS.2024.3375882
2. T. Koide, N. Fukushi, H. Nakano, and D. Chiba, "ChatSpamDetector: Leveraging large language models for effective phishing email detection," in *Proc. SecureComm 2024*, Dubai, UAE. arXiv:2402.18093.
3. G. Desolda, F. Greco, and L. Viganò, "APOLLO: A GPT-based tool to detect phishing emails and generate explanations that warn users," arXiv:2410.07997, 2024.
4. R. Nair, F. H. Abbasi, and S. Pervez, "PhishEmailLLM: A meta model approach to detect phishing emails by leveraging LLMs and machine learning models," in *Proc. ACSW 2025*, Brisbane, Australia. doi: 10.1145/3727166.3727169
5. "Evaluating the effectiveness of LLMs versus ML in identifying and detecting phishing email attempts," *Algorithms*, vol. 18, no. 10, 599, 2025. doi: 10.3390/a18100599
6. L. Schöni et al., "It's a match: Enhancing the fit between users and phishing training through personalisation," in *Proc. CHI 2025*, Yokohama, Japan. doi: 10.1145/3706598.3713845
7. L. Schöni et al., "You know what? Evaluation of a personalised phishing training," in *Proc. EuroUSEC 2024*. doi: 10.1145/3688459.3688460
8. K. Sriganthan and F. Härer, "An adaptive phishing awareness training environment based on LLMs and interactive learning," in *Proc. IEEE RTSI 2025*. doi: 10.1109/RTSI64020.2025.11212360
9. "TrainShield: Targeted awareness for cybersecurity training," in *Proc. ACM Hypertext (HT) 2025*. doi: 10.1145/3800935.3830870
10. F. Greco et al., "Improving phishing resilience with AI-generated training," arXiv:2512.01893, 2025.
11. "Supporting the design of phishing education, training and awareness with LLMs (DAMOCLES)," in *Proc. CEUR Workshop (HAI4CSS)*, vol. 3700, 2024.
12. L. Zhou and A. Aldkheel, "Gamification-based spear phishing awareness training system," in *Proc. AMCIS 2025*.

Appendix A: Demo Script (3 minutes)

1. Landing (30 s): one server, topbar, hero, feed, employees.
2. Detection (45 s): Simulate Attack → click row → walk 5 inspector steps.
3. Support (60 s): OTP question → Steps, Voice, Video tabs.
4. Devices (30 s): agent heartbeats, online/offline rule.
5. Proof (15 s): pytest 3 passed; /docs API.

Appendix B: API Table

Method	Endpoint	Purpose
GET	/health	Liveness probe
GET/POST	/api/employees	Directory list / add
POST	/api/threats	Detect + save + broadcast
GET	/api/threats, /api/stats, /api/perf	Feed + graphs
POST	/api/support/ask	Guided fix + media
POST/GET	/api/devices/*	Agent register, heartbeat, list
WS	/ws/alerts	Live push channel